

PROGRAMACIÓN PROCEDURAL



Unidad 4

Recursividad

Recursividad

- ▶ La recursividad es una alternativa a la iteración, es un proceso mediante el cual se puede definir una función en términos de sí misma.
- ▶ La recursividad en programación es una técnica fundamental que permite a una función llamarse a sí misma para resolver un problema.
- ▶ (+) La solución recursiva por ser más simple y natural que la solución iterativa correspondiente, permitiendo así, reducir su complejidad y ocultar detalles de programación.
- ▶ (-) Generalmente, la solución recursiva a un problema planteado suele resultar menos eficiente en cuanto al tiempo de ejecución y al almacenamiento en memoria.
- ▶ Es importante tener en cuenta que un procedimiento o una función recursiva (subprograma recursivo) debe cumplir con dos propiedades generales para no dar lugar a un ciclo infinito con las sucesivas llamadas:
 - Cumplir una cierta condición o criterio base del que dependa la llamada recursiva.
 - Cada vez que el procedimiento o función se llamen así mismos, directa o indirectamente debe estar más cerca del incumplimiento de la condición de que depende la llamada.

Tipo de recurrencias

Directa. Este tipo de recursividad es la más común y se presenta cuando una función se llama así misma una o varias veces.

Enunciado: Calcular el factorial de un número natural.

```
int factorial (int i )
{
    if ( i == 0 )
        return ( 1 )
    else
        return ( i * factorial ( i - 1 ) );
}
```

Indirecta. Este tipo de recursividad se presenta cuando una función es llamada de manera indirecta, es decir, a través de otra función, generando llamadas recursivas.

Enunciado: Realizar un programa que solicite un número y muestre en pantalla si el número es par o impar.

```
int par(int n)
{
    if (n == 0)
        return 1;
    return impar(n-1);
}

int impar(int n)
{
    if (n == 0)
        return 0;
    return par(n-1);
}
```

Recursividad

Es una alternativa a la iteración, un proceso mediante el cual se puede definir una función en términos de sí misma

Ej : Función Factorial

Definición Iterativa

$$\left\{ \begin{array}{ll} n! = n * (n - 1) * (n - 2) * \dots * 2 * 1 & \text{si } n > 0 \\ n! = 1 , & \text{si } n = 0 \end{array} \right.$$

Definición Recursiva

$$\left\{ \begin{array}{ll} n! = n * (n - 1)! & \text{si } n > 0 \\ n! = 1 , & \text{si } n = 0 \end{array} \right.$$

Factorial de n

$$n! = n * (n - 1)!$$



$$(n - 1) * (n - 2)!$$



$$(n - 2) * (n - 3)!$$

:



$$3 * 2!$$



$$2 * 1!$$

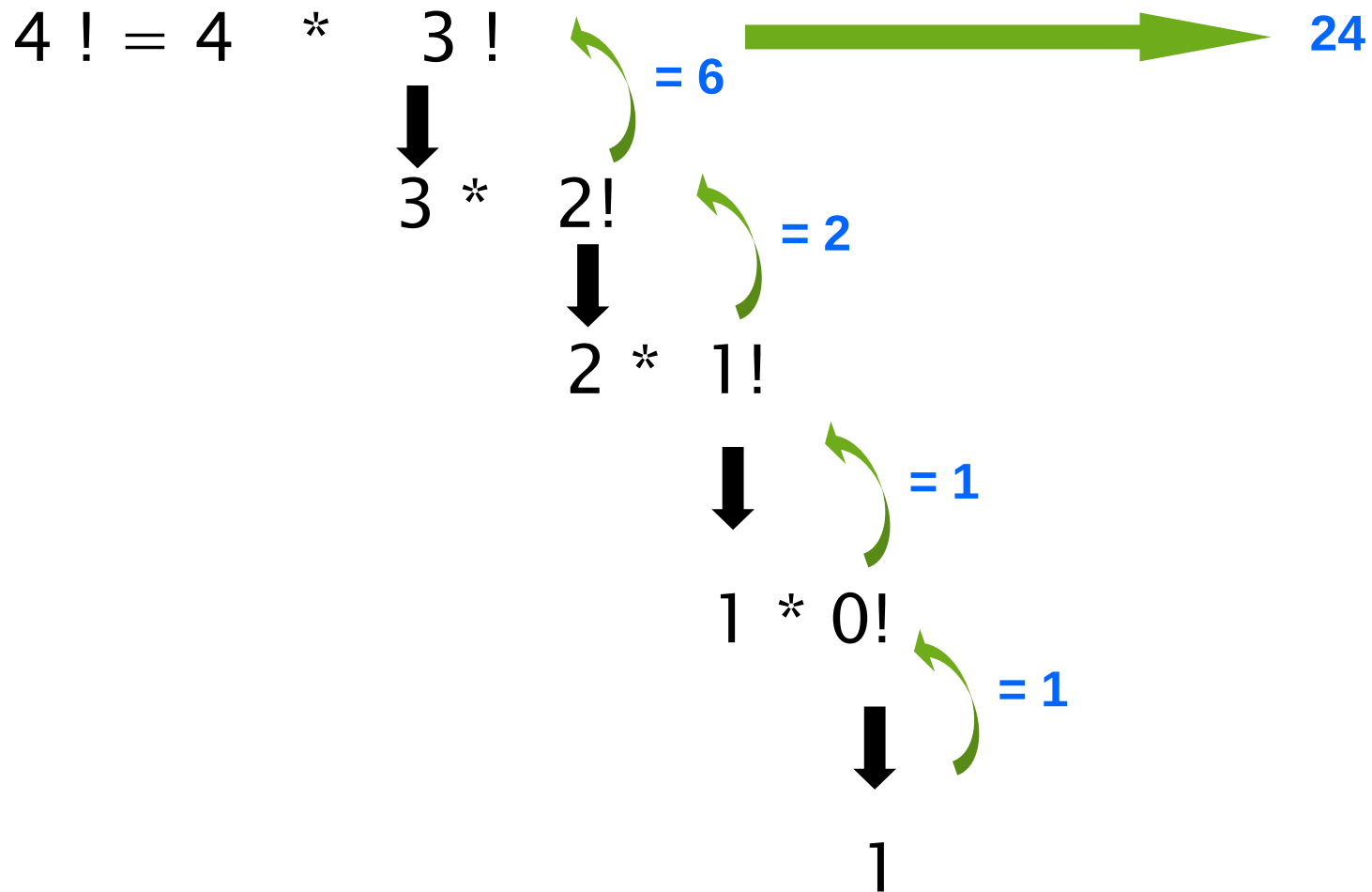


$$1 * 0!$$



1

Ejemplo: Factorial de 4

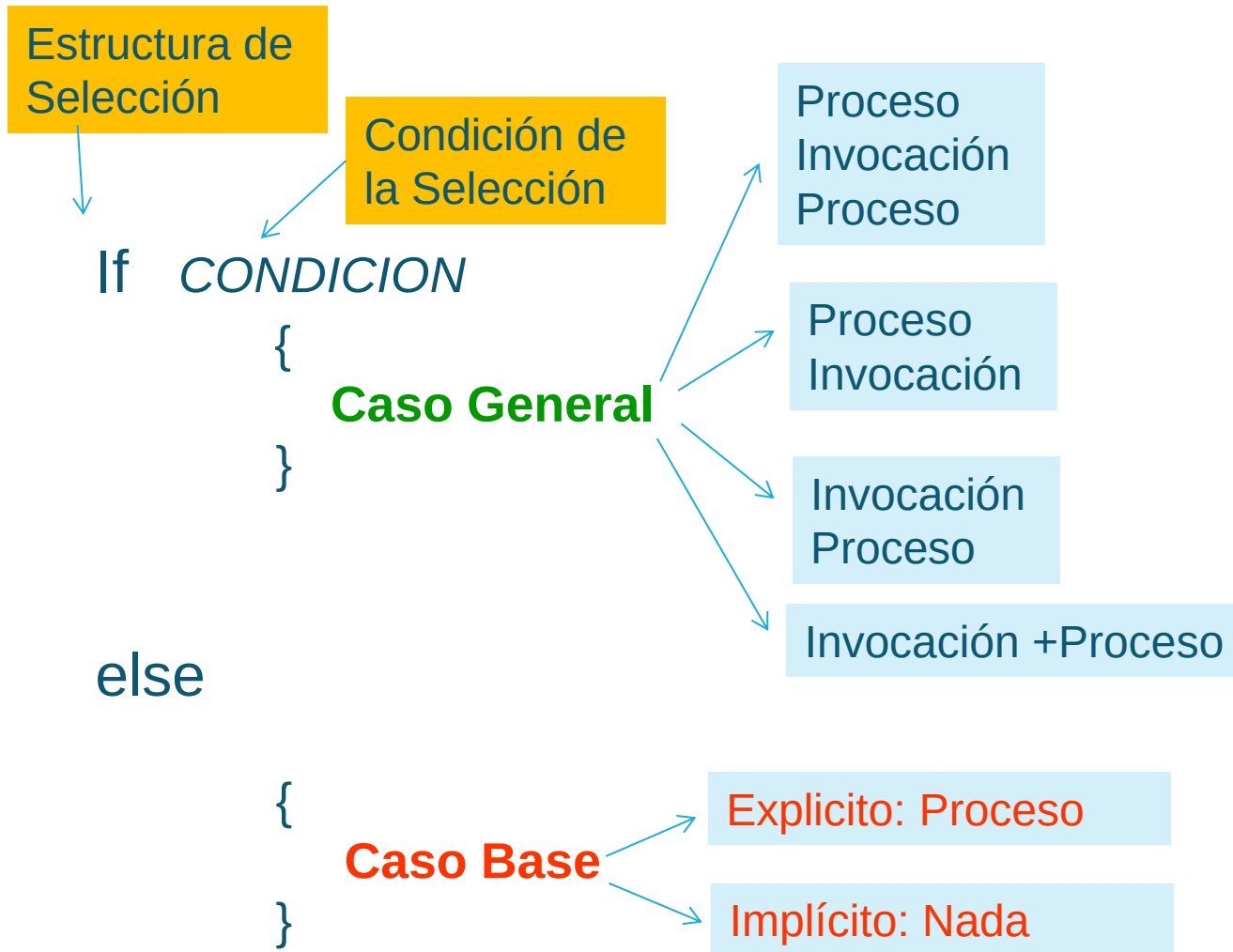


Construcción y ejecución de funciones recursivas

Al construir una función recursiva, se debe asegurar la existencia de una estructura de selección que permita identificar:

1. **Caso base (puede haber más de uno):** permite detener la invocación sucesiva de la función (no la vuelve a invocar); caso contrario se tendrían una serie infinita de invocaciones sucesivas. Es un conjunto de sentencias que se ejecutan cuando la recursión termina (entre las cuales no se encuentra una invocación de la función a si misma)
2. **Caso general (puede haber más de uno):** permite que la función se invoque a sí misma con valores de parámetros que cambian en cada llamada; acercándose cada vez más al caso base. Es un conjunto de sentencias que se ejecutan cuando la recursión continua (entre las líneas de sentencias esta una invocación de la función a si misma)

Estructura de función Recursiva



Construcción y ejecución de funciones recursivas

En la ejecución, las sentencias *que aparecen después de cada invocación no se resuelven inmediatamente, éstas quedan pendientes.*

Cada invocación recursiva, genera un registro de activación y se apila en el stack o pila.

Esos registros de activación se van desapilando en el orden inverso a como fueron apilados.



Factorial de 3

$n! = n * (n - 1)!$ si $n > 0$
 $n! = 1$, si $n = 0$

```
long int factorial ( int i )
```

```
{
```

```
if ( i == 0 )
```

```
    return 1; ➡ Caso Base
```

```
else
```

```
    return i * factorial ( i - 1 ) ; ➡ Caso General
```

```
}
```

```
main ( )
```

```
{
```

```
    int n;
```

```
    scanf ( " %d",& n);
```

```
    printf ( "\n n! = %l ", factorial ( n ) );
```

```
}
```

If *CONDICION*

{

Caso General

}

else

{

Caso Base explícito

}

Mapa de memoria

n: 3

factorial(3)

R1

i: 3

area temporal : 3 -1

resultado: i * factorial (2)

direcc. de retorno al (main)

```
long factorial ( long int i )
{
    if ( i == 0 )
        return 1
    else
        return (i * factorial ( i-1 ));
}

int main ( )
{
    int n;
    scanf (" %d",& n);
    printf (" n! = %d ", factorial(n ));
}
```

Mapa de memoria

n: 3

factorial(3)

R1

i: 3

area temporal : 3 -1

resultado: i * factorial (2)

direcc. de retorno al (main)

R2

i: 2

area temporal : 2 -1

resultado: i * factorial (1)

direcc. de retorno al punto de invocación (R1)

```
long factorial ( long int i )
{
    if ( i == 0 )
        return 1
    else
        return (i * factorial ( i-1 ));
}

int main ( )
{
    int n;
    scanf (" %d",& n);
    printf (" n! = %d ", factorial(n ));
}
```

Mapa de memoria

n: 3

factorial(3)

R1

i: 3

area temporal : 3 -1

resultado: i * factorial (2)

direcc. de retorno al (main)

R2

i: 2

area temporal : 2 -1

resultado: i * factorial (1)

direcc. de retorno al punto de invocación (R1)

R3

i: 1

area temporal : 1 -1

resultado: i * factorial (0)

direcc. de retorno al punto de invocación (R2)

i

```
long factorial ( long int i )
{
    if ( i == 0 )
        return 1
    else
        return (i * factorial ( i-1 ));
}

int main ( )
{
    int n;
    scanf ("%d",& n);
    printf (" n! = %d ", factorial(n ));
}
```

Mapa de memoria

n: 3

factorial(3)

R1

i: 3

area temporal : 3 -1

resultado: i * factorial (2)

direcc. de retorno al (main)

R2

i: 2

area temporal : 2 -1

resultado: i * factorial (1)

direcc. de retorno al punto de invocación (R1)

R3

i: 1

area temporal : 1 -1

resultado: i * factorial (0)

direcc. de retorno al punto de invocación (R2)

R4

i: 0

resultado: 1

direcc. de retorno al punto de invocación (R3)

6

2

1

1

```
long factorial ( long int i )
{
    if ( i == 0 )
        return 1
    else
        return ( i * factorial ( i-1 ));
}

int main ( )
{
    int n;
    scanf ( " %d",& n);
    printf ( " n! = %d ", factorial(n ));
}
```

eje2_op2.cpp activ1.CPP factorial_rekurs.CPP

```
1 #include<stdio.h>
2 #include<conio.h>
3 long factorial(long &n); /* prototipo de la función factorial */
4 // long factorial(long n) ;
5 main()
6 {
7     int n;
8     printf("\n n = ");
9     scanf(" %d", &n);
10    printf("\n n! = %d\n", factorial(n));
11    printf("\n valor del numero despues de la función n! = %d\n", n);
12    getch();
13    /*getchar();*/
14 }
15 long factorial(long &i)
16 // long factorial(long i)
17 { printf("\n firección de la variable en cada invocación %x\n", &i);
18     if(i==0)
19         return(1);
20     else
21         return(i*factorial(i-1));
22 }
23
```

D:\IVO\PROCEDURAL 2017\Ejercicios c\recursividad\factorial_rekurs.exe

```
n = 4
firección de la variable en cada invocación 12ff4c
firección de la variable en cada invocación 12ff3c
firección de la variable en cada invocación 12ff28
firección de la variable en cada invocación 12ff14
firección de la variable en cada invocación 12ff00
n! = 24
valor del numero despues de la función n! = 4
```

Sumar recursivamente los números pares menores o iguales que un valor ingresado por teclado.

Señalar Caso Base y General

Mostrar el estado de memoria para num=7

```
int acumula_pares ( int xnum)
{   if (xnum)
        if ((xnum % 2)== 0)
            return xnum + acumula_pares (xnum - 2);
        else return acumula_pares (xnum - 1);
    else return (0);
}

int main()
{   int num;
    scanf("%d", &num);
    printf("\n suma de pares menores que:  %d es  %d ", num,
acumula_pares(num));
    getch();
}
```


REGISTROS
APILADOS
acumula - pares

Registro de activación del main

```
int acumula_pares ( int xnum)
{
    if (xnum)
        if ((xnum % 2) == 0)
            return xnum + acumula_pares( xnum - 2);
        else return acumula_pares( xnum - 1);
    else return (0);
}
```

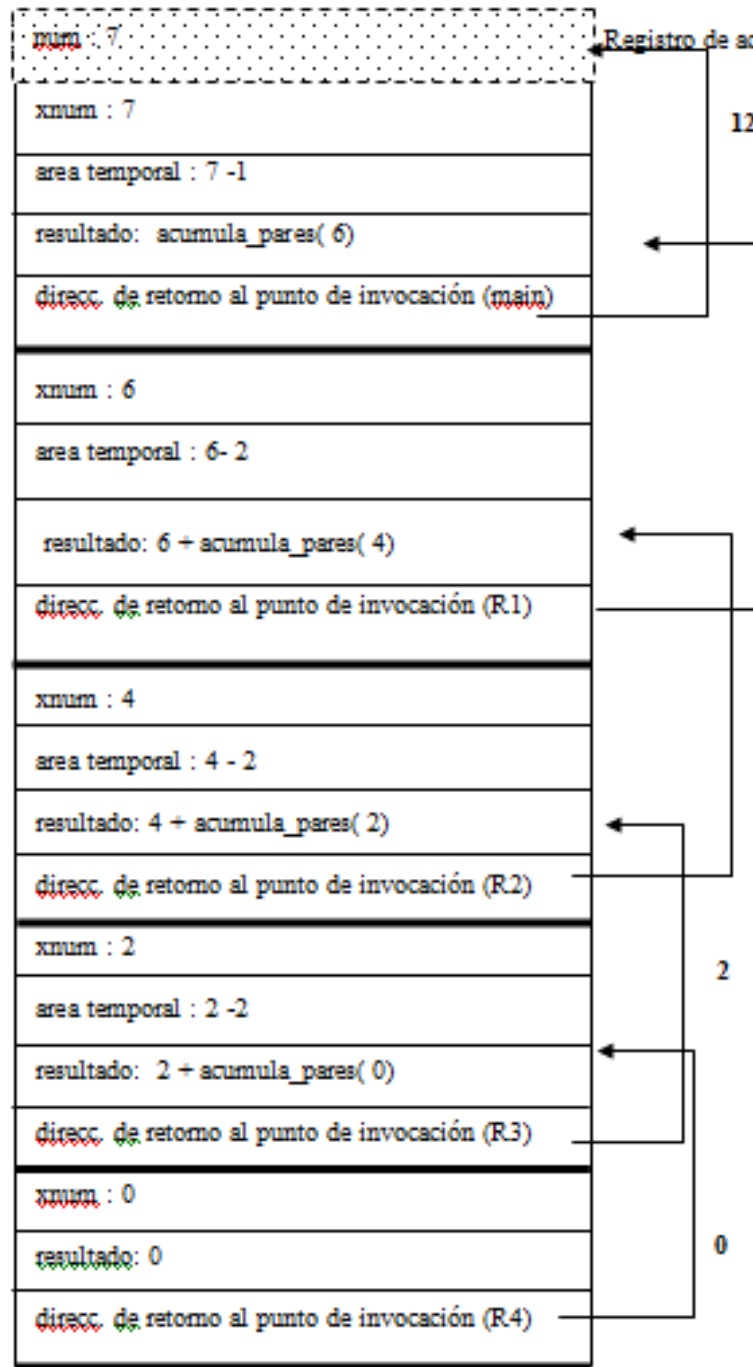
R1

R2

R3

R4

R5



12

12

6

2

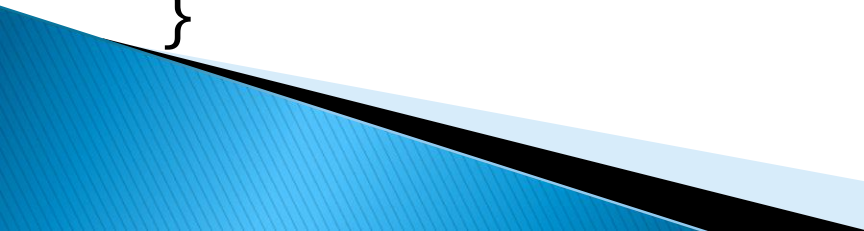
0

```
int main()
{
    int num;
    scanf("%d", &num);
    printf("\n suma de pares menores que: %d es %d ", num, acumula_pares(num));
    getch();
}
```

Dada la siguiente definición de una función, se pide:

- a) Construir la función main con la invocación.
- b) Señalar caso base y general.
- c) Graficar el mapa de memoria al invocar a listar (3) y realizar el seguimiento.

```
void listar ( int n)
{
    if ( n != 0 )
    { printf (" %d", n);
      listar ( n - 1 );
      printf (" %d", n); }
    else printf (" el número es cero");
    return;
}
```



```

#include <stdio.h>
#include <conio.h>

void listar( int n)
{
    if (n!=0)
    {
        printf (" \n %d", n);
        listar(n-1);
        printf (" \n %d", n);
    }
    else printf ("\n el número es cero");
    return;
}

int main(void)
{
    int a=3;
    listar(a);
    getch();
}

```

D:\IVO\PROCEDURAL 2015\Unidad 3\Ejerc

```

3
2
1
el n-mero es cero
1
2
3

```

Transformar un número a Binario

```
void Binario (int n)
```

```
{  
  if (n > 0)  
  {  
    Binario(n/2);  
    printf("%d", n % 2);  
  }  
}
```

```
void main()
```

```
{  
  int n;  
  printf("\n Introduzca un entero positivo: ");  
  scanf("%d", &n) ;  
  printf("\n El Numero Decimal %d en binario es ", n);  
  Binario(n);  
  printf(" en binario");  
}
```


$$\begin{array}{r} 13 \mid 2 \\ 1 \quad 6 \mid 2 \\ 0 \quad 3 \mid 2 \\ 1 \quad 1 \mid 2 \\ 1 \quad 0 \end{array}$$

- a) Señalar caso base y general
- b) Construir el Mapa de Memoria

```
#include <conio.h>
#include <stdio.h>

void Binario (int n)
{
    if (n > 0)
    {
        Binario(n/2);
        printf("%d", n % 2);
    }
}

int main()
{
    int n;
    printf("\n Introduzca un entero positivo: ");
    scanf("%d", &n) ;
    printf("\n El Numero Decimal %d en binario es ",
    Binario(n);
    printf(" en binario");
    getch();
}
```

D:\IVO\PROCEDURAL 2015\Ejercicios c\recursividad Resueltos\binario2.exe

Introduzca un entero positivo: 13

El Numero Decimal 13 en binario es 1101 en binario

Funciones recursivas que devuelven más de un resultado

Sumar recursivamente los números pares e impares menores o iguales que un valor ingresado por teclado. Realizar seguimiento y mapa de memoria para num=4

```
int acumula_pares ( int xnum)
{ if (xnum)
    if ((xnum % 2)== 0)
        return xnum + acumula_pares( xnum - 2);
    else return(acumula_pares( xnum - 1));
else return (0);
}

int main()
{ int num;
  scanf("%d", &num);
  printf("\n suma de pares menores que:  %d es  %d ", num, acumula_pares(num));
}
```

¿Qué modificaciones deberían realizarse al algoritmo utilizado anteriormente ?

	num : 4 impares: 0	imp
	xnum : 4	
R1	área temporal : 4 -1	
	resultado: xnum+ acu.. (3,imp)	
	direcc. de retorno al main	
R2	xnum : 3	
	área temporal : 3 -1	
	resultado:	
	direcc. de retorno al punto de invocación (R1)	
R3	xnum : 2	
	area temporal : 2 -1	
	resultado:	
	direcc. de retorno al punto de invocación (R2)	
R4	xnum : 1	
	area temporal : 1 -1	
	resultado:	
	direcc. de retorno al punto de invocación (R3)	
R5	xnum : 0	
	resultado: 0	
	Resultdo	
	direcc. de retorno al punto de invocación (R4)	

```

int acumula_pares_impares ( int xnum, int &imp)
{
    if (xnum)
        if ((xnum % 2)== 0)
            return (xnum + acumula_pares_impares ( xnum - 1,imp));
        else { imp+= xnum;
                return(acumula_pares_impares ( xnum - 1,imp));
            }
    else return (0);
}

```

```

int main(void)
{
    int num;
    int impares = 0;
    printf("\n ingrese un numero");
    scanf("%d", &num);
    printf("\n la suma de los números
           pares <=  %d ", num);

    printf("es : %d:",
           acumula_pares_impares (num,impares));
    printf("\n impares es : %d:", impares);
    getch();
}

```

```

#include <stdio.h>
#include <conio.h>
int acumula_pares_impares ( int xnum, int &imp)
{
    if (xnum)
        if ((xnum % 2) == 0)
            return xnum + acumula_pares_impares ( xnum - 1, &imp);
        else { imp += xnum;
            return(acumula_pares_impares ( xnum - 1, &imp));
        }
    else return (0);
}

int main()
{
    int num, impares=0;
    printf("\n Ingrese numero");
    scanf("%d", &num);
    printf("\n suma de pares menores que: %d es %d", num, impares);
    printf("\n suma de impares impares es : %d:", impares);

    getch();
}

```

D:\IVO\PROCEDURAL 2015\Unidad 3\Ejercicios teorí

Ingrese numesro4

suma de pares menores que: 4 es 6
 suma de impares impares es : 4:

Recursividad en Arreglos

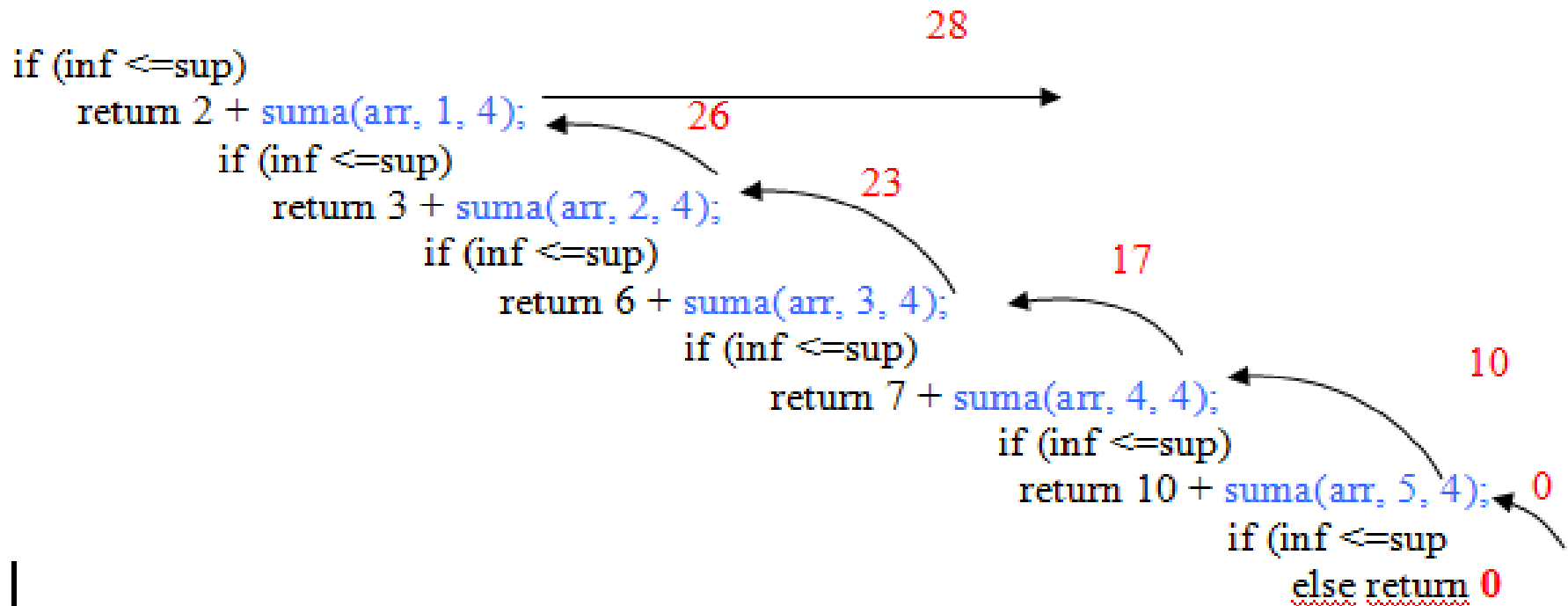
Suma recursiva de las componentes de un arreglo

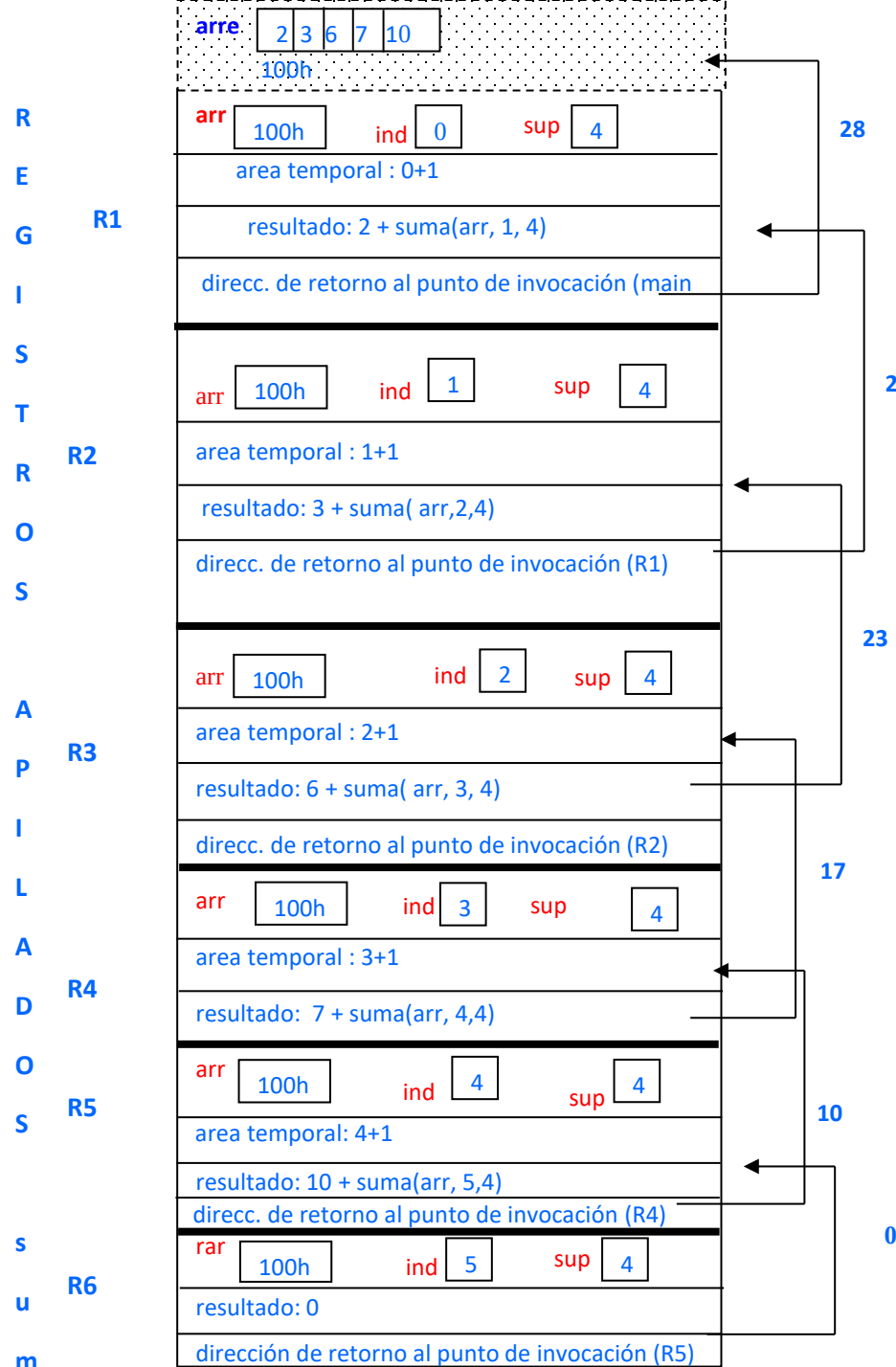
`int arre[5]={2, 3, 6, 7, 10}`

arre

2	3	6	7	10
---	---	---	---	----

Esquema de la solución:





```
#include<stdio.h>
#include<conio.h>
```

```
int suma(int arr[], int ind, int sup)
{
    if (ind <=sup)
        return arr[ind] + suma(arr, ind+1, sup);
    else return 0;
}

int main()
{
    int arre[5]={2, 3, 6, 7, 10} ;
    printf("\n Suma de  componentes %d",
        suma(arre, 0,4));

    getch();
}
```

Recursividad en Arreglos

```
int suma(int arr[], int ind, int sup)
{
    if (ind <=sup)
        return arr[ind] + suma(arr, ind+1, sup);
    else return 0;
}
```

```
int main()
{
    int arre[5]={2, 3, 6, 7, 10} ;
    printf("\n Suma de las componentes %d", suma(arre, 0,4));
    getch();
}
```

Modificar el algoritmo anterior de modo que la función devuelva, la suma de las componentes y el total de componentes mayores a 5. Señalar caso base y caso general

Recursividad en Arreglos

Calcular el producto escalar de los dos vectores y dado un valor, indicar en cual de los arreglos se encuentra.

```
void main(void)
{
    int a[n],b[n],valor;
    printf ("\n CARGA PRIMER VECTOR \n");
    carga(a,0);
    printf ("\n CARGA SEGUNDO VECTOR ");
    carga(b,0);
    printf ("\n Ingrese valor a buscar en los vectores: ");
    scanf("%d",&valor);
    if (busq_sec_rec(a,0,valor)==1)
        printf("\n valor está en  primer vector");
    else printf(" NO está en  primer vector");
    if (busq_sec_rec(b,0,valor)==1)
        printf("\n está en  segundo vector");
    else printf("\n No esta en  segundo vector ");
    printf ("\n Producto escalar de los vectores = %d", escalar(a,b,n -1));
}
```

La función recursiva **carga** es reusable, permiten cargar **arreglos de n componentes enteras positivas**.

La función **búsqueda** también es reusable.

Recursividad en Arreglos

```
#define n 5
void carga (int arr[n], int i)
{
    if (i!=n)
    { printf ("\n ingrese valor la posición %d:", i);
      scanf("%d",arr+i);
      carga(arr, i+1);
    }
}
```

```
int escalar (int x[n],int y[n],int j)
{
    if (j >= 0)
        return x[j]*y[j]+ escalar(x,y,j-1);
    else return 0;
}
```

```
int busq_sec_rec (int arr[n], int xi, int elem)
{
    if( xi==n )
        return -1;
    else if (arr[xi]==elem)
        return 1;
    else return busq_sec_rec ( arr, xi+1,elem);
}
```

Realizar el seguimiento y el mapa de memoria cuando se ejecuta la función **muestra**.

Señalar caso base y general

```
void muestra(int arr[], int ind, int sup)
{
    if (ind <= sup)
    {
        muestra(arr, ind+1, sup);
        printf("\n %d", arr[ind]);
    }
}
```

```
int main(void)
{
    int arre[5]={2, 3, 6, 7, 10};
    muestra(arre, 0,4);
    getch();
}
```



Recursión versus Iteración

Ambas se basan en una estructura de control:

- ❑ la iteración utiliza una estructura de repetición
- ❑ la recursión utiliza una estructura de selección.

Ambas implican repetición:

- ❑ la iteración utiliza la repetición de manera explícita
- ❑ a recursión consigue la repetición mediante repetidas llamadas a una misma función.

Ambas involucran una prueba de terminación:

- ❑ la iteración termina cuando falla la condición de continuación del ciclo
 - ❑ la recursión termina cuando se reconoce un caso base.
- 